```
import pandas as pd
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("saurabh00007/diabetescsv")

print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/saurabh00007/diabetescsv/1.arc
100%|██████████| 8.91k/8.91k [00:00<00:00, 12.7MB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/saurabh00007/diabetesc
```

```
import os
df = pd.read_csv(os.path.join(path, 'diabetes.csv'))
```

```
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Diabetes
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

Next steps: Generate code with df New interactive sheet

```
df.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
df.shape
```

```
(768, 9)
```

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
df.columns
```

```
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')
```

Start coding or generate with AI.

Replace Invalid Zeros with Median

```
cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']

for col in cols:
    df[col] = df[col].replace(0, df[col].median())
```

```
print(df['Outcome'].value_counts())
```

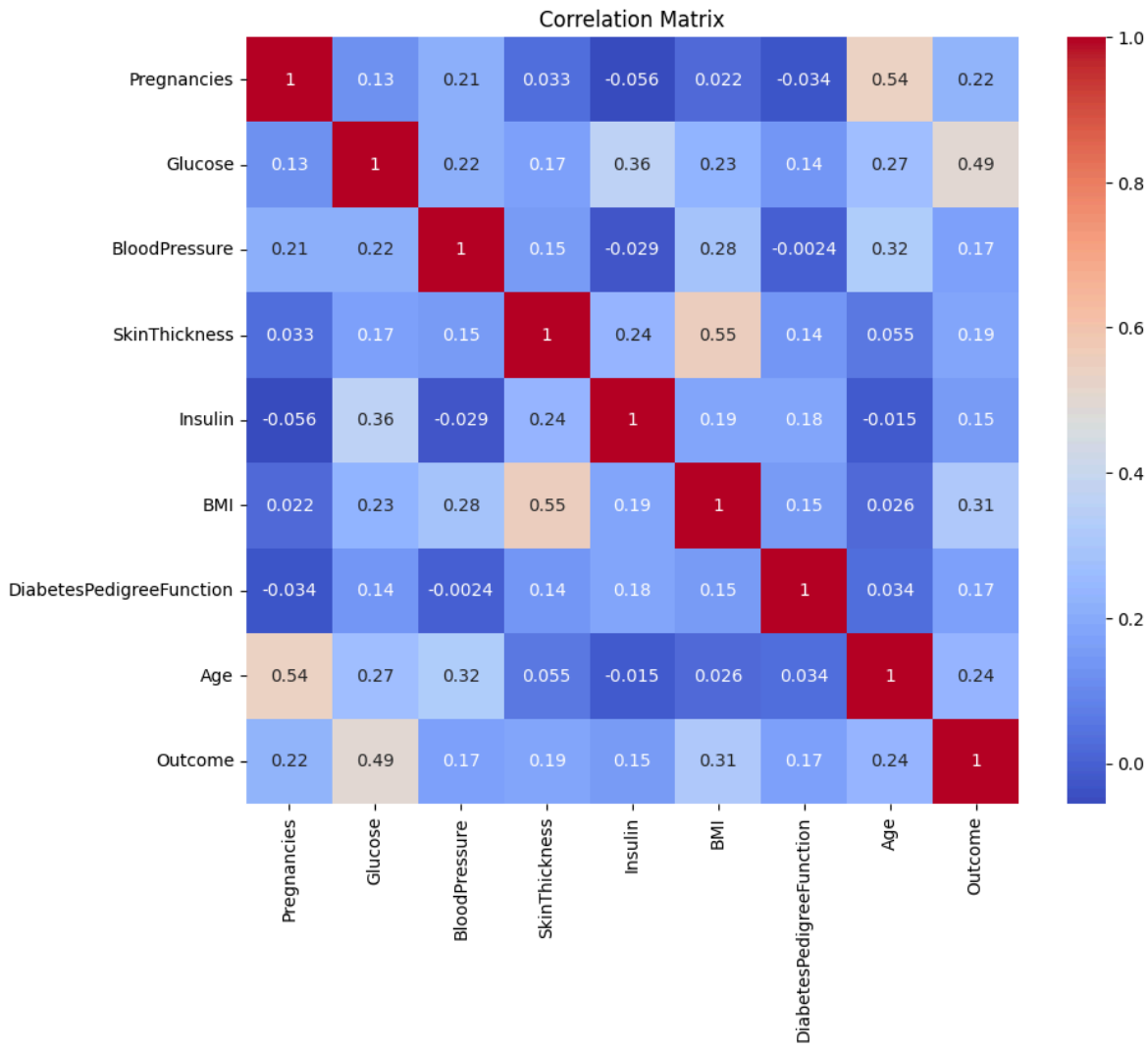
```
Outcome
0    500
1    268
Name: count, dtype: int64
```

Correlation Heatmap

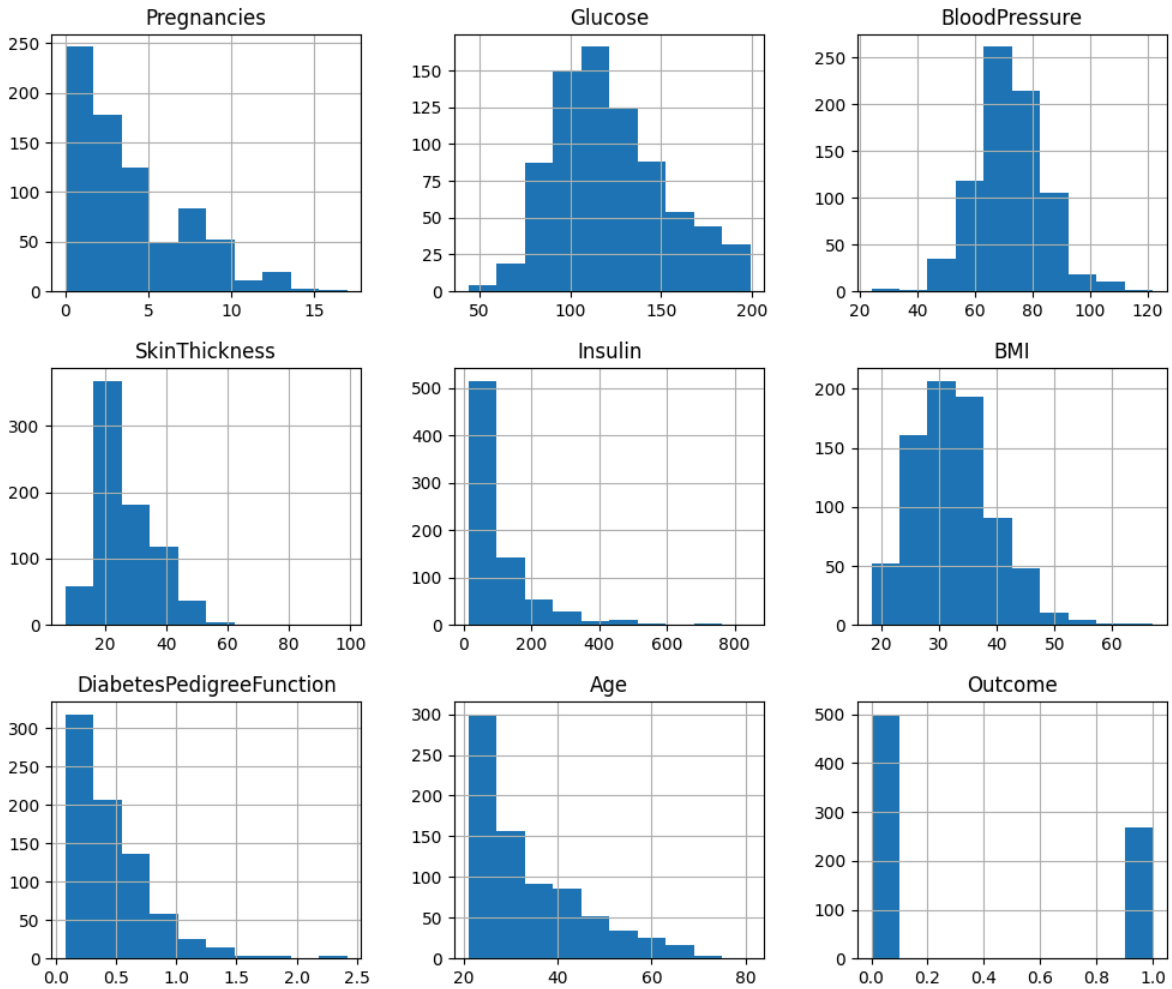
```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))
```

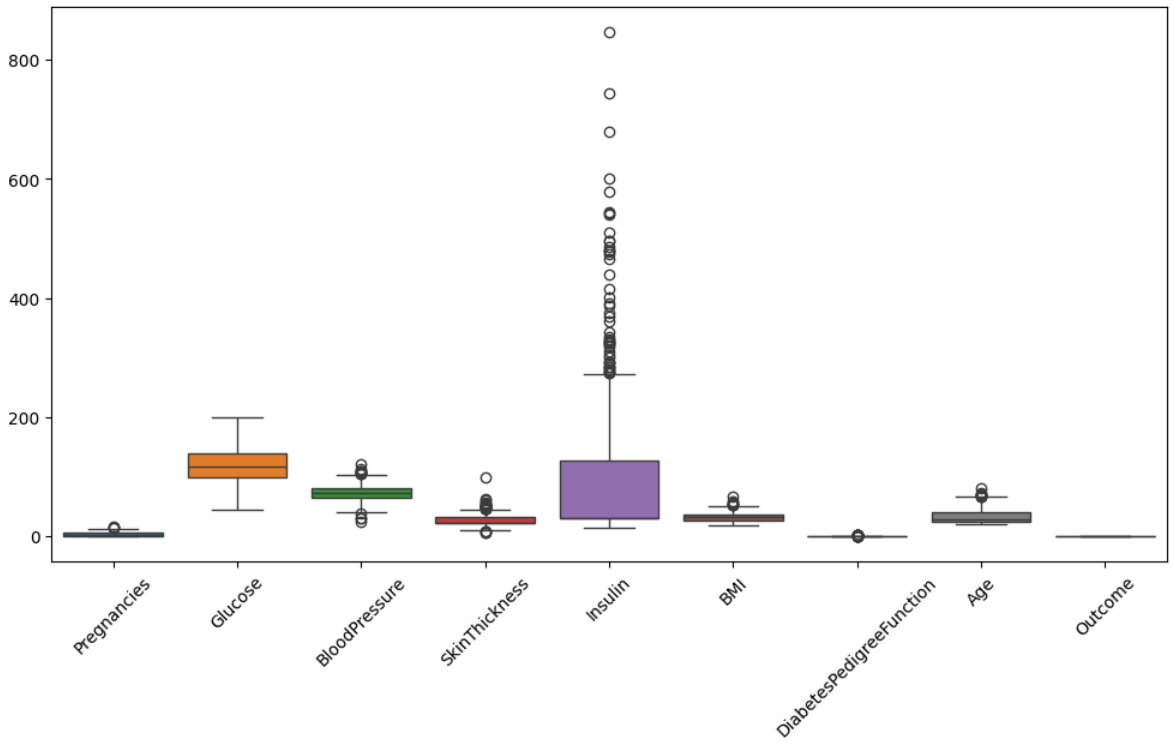
```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```




```
df.hist(figsize=(12,10))
plt.show()
```




```
plt.figure(figsize=(12,6))
sns.boxplot(data=df)
plt.xticks(rotation=45)
plt.show()
```



Start coding or generate with AI.

Split Data

```
from sklearn.model_selection import train_test_split

X = df.drop('Outcome', axis=1)
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Start coding or generate with AI.

Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Train KNN Model

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

Predictions

```
y_pred = model.predict(X_test)
```

Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7532467532467533
[[80 19]
 [19 36]]
              precision    recall  f1-score   support

           0       0.81      0.81      0.81        99
           1       0.65      0.65      0.65        55

    accuracy                           0.75       154
   macro avg       0.73      0.73      0.73       154
weighted avg       0.75      0.75      0.75       154
```

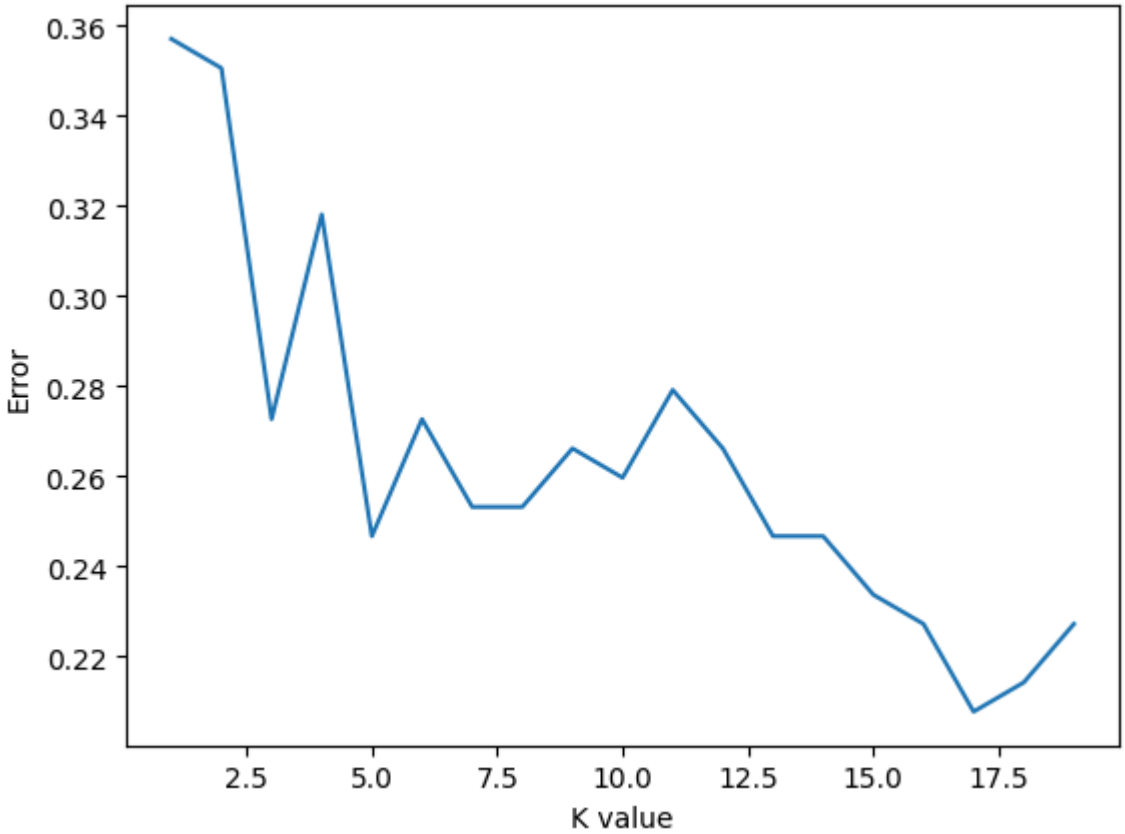
Start coding or generate with AI.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
error = []

for k in range(1, 20):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    pred = knn.predict(X_test)
    error.append(np.mean(pred != y_test))

plt.plot(range(1,20), error)
plt.xlabel("K value")
plt.ylabel("Error")
plt.show()
```




```
sample = [[2, 120, 70, 20, 79, 25.0, 0.5, 30]]

sample_scaled = scaler.transform(sample)

prediction = model.predict(sample_scaled)

# Output
if prediction[0] == 1:
    print("Patient is Diabetic")
else:
    print("Patient is Not Diabetic")
```

```
Patient is Not Diabetic
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Use
```

```
  warnings.warn(
```

Start coding or generate with AI.

Start coding or generate with AI.

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.